

GitHub Issues로 업무 관리하기

해야 할 일을 기록하고, 담당자를 정하고, PR로 완료하기

Issue란?

프로젝트에서 추적해야 할 작업이나 논의를 기록하는 단위다.

- 버그 제보
- 새로운 기능
- 문서 수정
- 리팩터링
- 조사 또는 의사결정

할 일 발견 → Issue 생성 → 담당/분류 → Branch 작업
→ PR 생성 → Review → Merge → Issue 종료

좋은 Issue의 조건

이슈만 읽어도 다음 질문에 답할 수 있어야 한다.

1. **무엇이** 문제이거나 필요한가?
2. **왜** 해야 하는가?
3. **어디까지** 하면 완료인가?
4. 참고할 화면, 로그, 문서가 있는가?

큰 기능은 작은 이슈로 나누고, 한 이슈에는 하나의 완료 목표를 둔다.

제목 작성법

제목은 목록에서 작업을 구분할 수 있도록 구체적으로 작성한다.

아쉬운 제목	개선한 제목
로그인 오류	로그인 실패 시 오류 메시지가 표시되지 않음
UI 수정	모바일에서 주문 버튼 너비 수정
문서 추가	README에 로컬 실행 방법 추가

팀에서 접두사를 정해도 좋다.

[Bug] 로그인 실패 메시지 누락
[Feature] 비밀번호 재설정 기능 추가
[Docs] 개발 환경 설정 안내 추가

본문 작성 예시

배경

로그인 실패 원인을 사용자가 확인할 수 없습니다.

작업 내용

- 실패 응답의 메시지를 화면에 표시
- 이메일 입력값은 유지

완료 조건

- [] 잘못된 비밀번호 입력 시 안내 문구가 보임
- [] 로그인 성공 동작에 영향이 없음

참고

- 관련 화면 또는 로그 첨부

체크박스는 진행 상황과 완료 기준을 함께 보여 준다.

Assignee: 누가 담당하는가?

Assignee는 이슈를 실제로 처리할 담당자다.

- 담당자가 정해지면 책임과 문의 대상을 알 수 있다.
- 공동 작업이면 여러 명을 지정할 수 있다.
- 담당자가 아직 없으면 비워 두고 분배 회의에서 정한다.

권장 흐름:

이슈 등록 → 우선순위 확인 → 담당자 배정 → 작업 시작

이슈 작성자와 담당자는 같을 필요가 없다.

Label: 어떤 종류의 일인가?

Label은 이슈와 PR을 분류하고 검색하는 저장소 단위의 태그다.

분류 기준	예시
작업 유형	bug , feature , docs , refactor
우선순위	priority: high , priority: low
영역	frontend , backend , infra
상태 보조	blocked , needs-review

기준을 섞지 말고 분류: 값 형태로 이름을 통일하면 찾기 쉽다.

Label 운영 원칙

- 자주 검색하거나 묶어 볼 기준만 만든다.
- 이름과 설명만 보고 용도를 알 수 있게 한다.
- 같은 뜻의 Label을 중복 생성하지 않는다.
- 상태는 가능하면 Projects의 `Status` 로 관리한다.

검색 예시:

```
is:issue is:open label:bug  
is:issue is:open assignee:@me  
is:issue is:open label:"priority: high"
```


Milestone: 어떤 목표까지 묶는가?

Milestone은 같은 목표나 마감일을 가진 이슈와 PR의 묶음이다.

예시:

- v1.0 출시
- 1차 스프린트
- 중간 발표

Milestone에는 제목, 설명, 마감일을 설정할 수 있고
연결된 이슈와 PR의 완료 비율을 확인할 수 있다.

Label은 분류, Assignee는 담당자, Milestone은 목표/기한이다.

Issue 생성 순서

1. 저장소에서 Issues → New issue를 선택한다.
2. 행동이 드러나는 제목을 작성한다.
3. 배경, 작업 내용, 완료 조건을 본문에 적는다.
4. Assignee를 지정한다.
5. Label과 Milestone을 선택한다.
6. 관련 이슈나 문서를 링크하고 제출한다.

등록 후에도 범위와 담당자가 바뀌면 내용을 최신 상태로 수정한다.

Issue와 Branch 연결하기

브랜치 이름에 이슈 번호를 넣으면 작업의 출처를 찾기 쉽다.

```
feature/12-login-error-message  
fix/27-mobile-button-width  
docs/31-local-setup
```

커밋 메시지에도 이슈를 참조할 수 있다.

```
feat: 로그인 실패 메시지 표시 (#12)
```

#12 는 참조이며, 이 표현만으로 이슈가 자동 종료되지는 않는다.

Issue와 PR 연결하기

PR 본문에 종료 키워드와 이슈 번호를 작성한다.

```
## 관련 이슈  
Closes #12
```

사용 가능한 대표 키워드:

- `close` , `closes` , `closed`
- `fix` , `fixes` , `fixed`
- `resolve` , `resolves` , `resolved`

PR이 병합되면 연결된 이슈가 자동으로 닫힌다.

키워드	ню앙스	예시 상황
close , closes , closed	그냥 "이슈를 종료한다" (일반적)	기능 추가, 문서 작업, 버그가 아닌 작업 완료
fix , fixes , fixed	"버그를 고쳤다"	버그 리포트 이슈를 해결했을 때
resolve , resolves , resolved	"문제/요청을 해결했다"	질문, 개선 요청 이슈를 처리했을 때

종료 키워드의 중요한 조건

자동 종료는 PR이 저장소의 **기본 브랜치(Default Branch)** 를 대상으로 할 때 동작한다.

이번 실습 레포는 **dev** 를 기본 브랜치로 설정했다.

feature → dev(PR) → Merge → Issue 자동 종료
dev → main(PR) → Merge → Issue 자동 종료 안 됨 (이미 dev에서 종료됨)

- **dev** : 실제 작업 완료 기준점 (기본 브랜치 → 이슈 자동 종료)
- **main** : 검증된 코드의 최종 배포처 (기본 브랜치 아님 → 자동 종료 대상 아님)

주의: "기본 브랜치"는 이름이 **main** 이냐 **dev** 냐가 아니라, **Settings에서 지정된 브랜치** 를 의미한다. 실무에서는 관습적으로 **main** 을 기본 브랜치로 쓰는 경우가 많으니, 다른 프로젝트에 참여할 때는 반드시 Default branch 설정을 먼저 확인한다.

다른 저장소의 이슈 참조하기

같은 레포 안에서는 그냥 #12만 써도 되지만, 레포가 다르면 GitHub이 어떤 레포의 12번 이슈인지 알 수 없어요. 그래서 전체 경로를 써야한다.

예: sk-family/backend-api 레포의 PR에서
sk-family/frontend-web 레포의 이슈를 닫고 싶을 때

Fixes sk-family/frontend-web#12

Issue Template이 필요한 이유

자유 형식만 사용하면 필요한 정보가 자주 빠진다.

- 재현 방법이 없는 버그
- 완료 조건이 없는 기능 요청
- 환경 정보가 없는 오류 보고
- 사람마다 다른 제목과 본문 구조

Template은 작성자를 귀찮게 하는 양식이 아니라 **작업을 시작하기 위한 최소 정보를 확보하는 장치**다.

Issue Template 만들기

저장소에서 다음 경로로 이동한다.

```
Settings → Issues → Set up templates
```

또는 기본 브랜치에 직접 파일을 추가한다.

```
.github/  
└─ ISSUE_TEMPLATE/  
    └─ bug_report.md  
    └─ feature_request.md
```

Template은 기본 브랜치에 반영되어야 새 이슈 화면에 나타난다.

Report Template 예시

새로운 기능

```
---  
name: Feature request  
about: 새로운 기능을 제안해주세요  
title: "[Feature] "  
labels: enhancement  
assignees: ""  
---
```

배경

이 기능이 왜 필요한가요? 어떤 문제를 해결하나요?

제안 내용

어떤 기능을 원하시나요? 구체적으로 설명해주세요.

기대 효과

이 기능이 추가되면 무엇이 좋아지나요?

참고 자료

관련 이미지, 링크, 예시가 있다면 첨부해주세요.

오류

```
---  
name: Bug report  
about: 발견한 오류를 알려주세요  
title: "[Bug] "  
labels: bug  
assignees: ""  
---
```

현상

어떤 문제가 발생했나요?

재현 방법

1.

기대 결과

어떻게 동작해야 하나요?

실행 환경

OS, 브라우저, 앱 버전 등을 적어주세요.